Lab Assignment 2: Two-Pass Expression Translator (Infix → Postfix → Value)

Course: CS3104 Compiler Design Laboratory

Syllabus Unit: Unit 1 — Introduction to Compilers

Date: 06/08/2026

Instructions

- Write your name and roll number on top of your source code as comments.
- Source file name must be "Lab2.c".
- Submit only the source file at - http://10.22.11.15/~abyaym/2026/CS3104_26/submission/

Problem Statement

A compiler is not one giant function — it is a pipeline of phases, each producing a well-defined intermediate representation that the next phase consumes. An interpreter, in contrast, often does everything in one pass. In this assignment you will feel that difference directly by building your arithmetic evaluator as two independent phases instead of one, and by keeping the two phases machine-independent and machine-dependent respectively.

Phase 1 (front end, machine-independent): read an infix expression containing non-negative integers, the operators +, -, *, / and parentheses, and translate it into postfix (Reverse Polish) notation using the classic shunting-yard algorithm. This phase only needs to know about operator precedence and associativity — it never needs to know how the target machine adds two numbers.

Phase 2 (back end, machine-dependent): read the postfix form produced by Phase 1 and evaluate it using an explicit operand stack, exactly the way a stack-based virtual machine would execute compiled code.

Example

```
Input   :  3 + 4 * (2 - 1)
Postfix :  3 4 2 1 - * +      <-- printed by Phase 1
Result  :  7                 <-- printed by Phase 2
```

Core Requirements

- **Tokenizer:** split the input string into numbers, operators, and parentheses (numbers may have multiple digits; spaces are optional between tokens).
- **Phase 1 – toPostfix():** implement the shunting-yard algorithm with an operator stack; respect precedence (*, / higher than +, -) and left-associativity.
- **Phase 2 – evalPostfix():** evaluate the postfix token list with a numeric stack; detect division by zero.
- **Driver:** print the postfix form and then the final value, on separate lines, matching the example above.
- **Error detection:** mismatched parentheses and invalid characters must produce an error message instead of a crash or wrong answer.

Practice Problems

1. Trace by hand, then verify with your program: 8 + 2 * 5 - (1 + 1) . Confirm the postfix string your program prints matches your hand trace.

2. Feed your program an unbalanced expression such as (3 + 4 * 2 and confirm it reports an error rather than looping or crashing.

Implementation Guidelines

- Keep Phase 1 and Phase 2 in separate functions (even separate translation units if you like) — Phase 2 must not re-parse the original infix string.
- Think about why this two-pass structure would let you swap in a different back end (e.g. one that emits machine code) without touching Phase 1 at all — this is the machine-independent vs. machine-dependent split from the lecture.

Compilation & Testing

```
$ gcc Lab2.c -o lab1
$ ./lab2
Enter expression: 3 + 4 * (2 - 1)
```

Deliverables

- Lab2.c